

CEDT-Assignment-style Reference

Every macro and environment this style provides, with a worked example
XXXXXXXXXX Your Name

Contents

1 Math shortcuts	1
2 Problems and sub-problems	1
3 Theorem-style environments	2
4 Solution, proof, and to-submit boxes	2
5 Code blocks	3
6 Figures	3
7 Tables	4
8 Multiple choice	4
9 Spacing helper	4

1 Math shortcuts

Common number sets and delimiters get short macros so you don't have to remember the `amsmath` spelling every time:

- Sets: $\mathbb{R}, \mathbb{C}, \mathbb{Z}, \mathbb{N}$ – `\R \C \Z \N`
- Delimiters: $|x|, \|v\|, (a+b), [a,b]$ – `\abs \norm \paren \sqbracket`

For example: $\forall x \in \mathbb{R}, |x| \geq 0$, and $\|v\| = \sqrt{(v_1^2 + v_2^2)}$.

2 Problems and sub-problems

Problem 1. problem auto-numbers itself. Sub-items nest under `subproblems` (decimal, e.g. 2.1) by default:

1.1 Decimal sub-items look like this.

1.2 A sub-item can go one level deeper with `Tsubsubproblems`:

T1.2.1 A "T"-prefixed third nesting level – adjust the fixed "T" in the style file if it doesn't fit your subject.

T1.2.2 Another one at the same depth.

Or letter-labelled with `subproblems_alpha` instead:

a) Like this.

b) And this.

Problem 10. Passing a number, e.g. `\begin{problem}[10]`, sets/reuses that exact problem number instead of auto-incrementing – handy when your numbering has to match a printed assignment sheet. Auto-numbering after this continues from whatever the highest-used number is, skipping any manually reused ones.

3 Theorem-style environments

Standard numbered `amsthm` environments are set up too – `theorem/lemma/proposition/corollary` share one counter, `definition` numbers separately, and `remark` is unnumbered:

Definition 3.1. An integer n is *even* if $n = 2k$ for some $k \in \mathbb{Z}$.

Theorem 3.1 (Fundamental theorem of arithmetic). *Every integer greater than 1 has a unique prime factorization, up to the order of factors.*

Lemma 3.2. *If n^2 is even, then n is even.*

Proposition 3.3. *The sum of two even integers is even.*

Corollary 3.4. *If n is even, then n^2 is even.*

Remark. These come straight from `amsthm` – nothing custom beyond the numbering/sharing scheme set up in the style file.

4 Solution, proof, and to-submit boxes

Problem 11. Worked-through answers that aren't the graded deliverable pair with `solution`.

Solution

The full derivation goes here.

A second paragraph indents correctly too – `CEDT-Assignment-style.sty` restores `\parindent` inside every breakable box below (`tcolorbox`'s `breakable` library, needed so long boxes can split across a page, zeroes it internally otherwise).

Problem 12. A problem that must be proven pairs with `proofbox`.

Proof

The proof goes here.

A second paragraph, indented the same way. □

Problem 13. A problem whose final answer must be called out for grading pairs with `tosubmit` – its corner tab reads "TO SUBMIT | Solution" rather than just "TO SUBMIT".

TO SUBMIT | Solution

This is what a grader should look at directly.

A second paragraph here indents too.

All three corner-tab labels shown so far – "Solution", "Proof", "TO SUBMIT | Solution" – are just each box's default; pass `title=...` through the optional argument to override one for a single use, no `.sty` edit needed (same mechanism as `codingbox`'s `[language=C++]` below):

FINAL ANSWER | Solution

A custom label via `\begin{tosubmit}[title={FINAL ANSWER | Solution}]`.

5 Code blocks

codingbox defaults to Python, but any language listings ships a `\lstdefinelanguage` for works via its language option:

```
1 print("Python is the default language")
```

```
1 #include <iostream>
2
3 int main() {
4     // Compute the first 10 Fibonacci numbers
5     int a = 0, b = 1;
6     for (int i = 0; i < 10; ++i) {
7         std::cout << a << " ";
8         int next = a + b;
9         a = b;
10        b = next;
11    }
12    return 0;
13 }
```

```
1 #!/usr/bin/env bash
2 # Print the first 10 Fibonacci numbers
3 a=0
4 b=1
5 for _ in $(seq 1 10); do
6     echo -n "$a "
7     next=$((a + b))
8     a=$b
9     b=$next
10 done
```

6 Figures

`\fig[width]{path}{caption}{label}` centers an image in a numbered, captioned, labelled figure float (width defaults to $0.7\text{\linewidth}$):

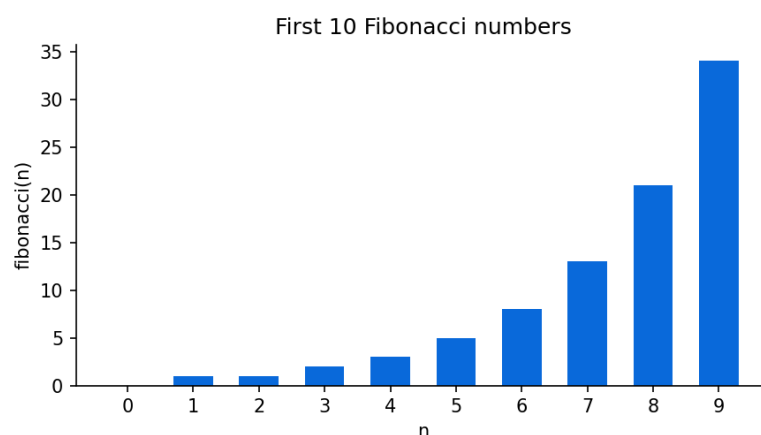
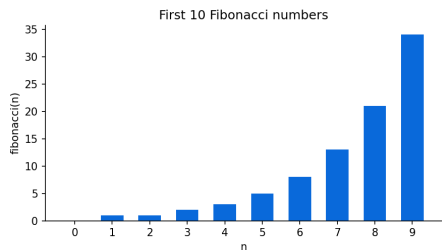


Figure 1: A numbered, captioned figure.

The starred form, `\fig*[width]{path}{}`, drops the caption/label entirely – just the centered image (the last two arguments are still required syntactically, but ignored, so pass `{}`):



7 Tables

`\begin{simpletable}{<colspec>}{<caption>}{<label>}` wraps a `booktabs`-style table with a numbered caption/label – write rows as normal and add `\midrule` after the header yourself:

<i>A</i>	<i>B</i>	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Table 1: Truth table for logical AND

8 Multiple choice

Problem 14. `\choices[<num choices>]{choice A, choice B, ...}` lays out options, auto-picking 4, 2, or 1 columns depending on how long the choices are (never more columns than there are choices):

- 14.1 Short choices sit four to a row.

A. TrueB. FalseC. MaybeD. Unknown
- 14.2 Longer choices wrap down to two columns automatically.

A. The probability increasesB. The probability decreasesC. The probability stays the sameD. Not enough information to tell
- 14.3 Pass the choice count explicitly when it isn't 4.

A. A B. B C. C D. D
E. E F. F

9 Spacing helper

`\myspace[hor v][amount][unit]` wraps `\hspace/\vspace`; all three arguments are optional and default to `\myspace = \vspace{4mm}`. For example, `A\myspace[h][1][cm]B` gives A B (a 1cm gap between A and B on the same line).

APPENDIX 1

notebook-template | Code Solution

notebook-template

September 2, 2026

1 Example Notebook

This notebook is a worked example of `templates/nbconvert/cedt-assignment/` — the shared `jupyter nbconvert` LaTeX template in this repo. Convert it yourself with:

```
make nb-pdf NB=templates/notebook-template.ipynb
```

(from a subject repo that vendors this template, with `course-config.tex` filled in) to see the same header, 9pt font, and GitHub-style code boxes as `templates/assignment-template.tex`.

1.1 Problem

Write a function that returns the n -th Fibonacci number (0-indexed), then print the first ten values.

```
[1]: def fibonacci(n):  
      """Return the n-th Fibonacci number (0-indexed)."""  
      a, b = 0, 1  
      for _ in range(n):  
          a, b = b, a + b  
      return a  
  
      # A short comment, to show off comment styling too  
      values = [fibonacci(n) for n in range(10)]  
      print("First 10 Fibonacci numbers:", values)
```

First 10 Fibonacci numbers: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

1.2 Notes

- Markdown cells render as normal LaTeX prose.
- Code cells get GitHub-style syntax highlighting via the `cedt-assignment` nbconvert template, matching `CEDT-Assignment-style.sty`'s `codingbox`.
- Both this source `.ipynb` and its compiled `.pdf` are committed here purely as a reference; nothing in this repo actually builds them itself (this repo isn't a subject repo — it has no `course-config.tex/vendor/` of its own).